SCITON · ENGINEERING REFERENCE

# Teensy 4.1 micro-ROS — performance

A Teensy 4.1 peripheral streaming to ROS2 (Jazzy) over micro-ROS, taken from a plain-serial baseline to a tuned Ethernet/UDP node. This characterizes the four axes that decide whether it can carry a real-time control and telemetry link: raw throughput, round-trip latency, the best-effort vs reliable QoS trade-off, and how the rate holds up when the host is busy. Every figure traces to a committed benchmark node or capture script.

**MCU** Teensy 4.1 (600 MHz Cortex-M7) · **Transport** QNEthernet / lwIP, UDP packet mode · **Host** ROS2 Jazzy · Ubuntu 24.04 / WSL2 (mirrored) · **Link** direct Gigabit · **Captured** June 2026

## Headline

The original goal was a peripheral that could publish above **500 Hz**. The tuned Ethernet/UDP node clears that by more than an order of magnitude, and the firmware is never the limiting factor — the receive pipeline (agent → DDS → subscriber) is.

| ~40 kHz | ~0.5 ms | 0.15 % | ≥ 40× |
|---|---|---|---|
| Lossless ceiling, 12 B `Point32` (C++ subscriber) | Round-trip latency (≈0.3 ms one-way), p99 < 1.2 ms | Drops at 1 kHz with 15/16 host cores pegged | Margin over the original >500 Hz target |

- **Lossless to ~40 kHz** for the 12-byte `Point32` payload and **~20–25 kHz** for the 28-byte multi-signal `GalvoState`, measured with a C++ subscriber on a direct Gigabit link.
- **Sub-millisecond round trip** — ~0.5 ms median RTT, ~0.3 ms one-way, p99 under 1.2 ms, 0% loss.
- **The bottleneck is the micro-ROS agent** (~43k msg/s, single-threaded XRCE → FastDDS), not the Teensy — the firmware emits its exact target rate up to its 50 kHz clamp.
- **Graceful under host load** — at 1 kHz the link is effectively bulletproof; degradation is a few percent of samples and some tail latency, not a rate collapse.
- **Two QoS profiles, by design** — best-effort for high-rate telemetry, reliable (zero-loss, ~2.8 kHz cap) for commands and setpoints.

## What's measured, and on what

One Teensy firmware publishes a monotonic sequence number in the payload's `z` field; the host meters detect drops as gaps in that sequence and report received rate over fixed windows. The node carries two publishers on the same firmware so each QoS can be isolated with `teensy/set_mode` (no cross-load), and the publish rate is changed live over `teensy/set_rate_hz` without a reflash. Payloads: `geometry_msgs/Point32` (12 B) and the custom multi-signal `teensy_msgs/GalvoState` (~28 B).


The measured path. The Teensy publishes on a `micros()` cadence over a QNEthernet custom UDP transport; the host meters (`teensy_bench`, C++) sit at the far end past the agent and DDS.

> Unless noted, all numbers are on a **direct Gigabit link** with a **C++ subscriber** (`teensy_bench rate_meter` / `rtt_ping`), 5-second windows, drop detection via the monotonic `.z` sequence. The host runs ROS2 Jazzy under WSL2 with mirrored networking. The C++ subscriber matters: a Python rclpy subscriber is itself a receive-side limiter (see the throughput section), so it understates the link.

PART 1 — THROUGHPUT & THE LOSSLESS CEILING

## True ceiling and payload-size effect

Best-effort, swept past the knee with the C++ subscriber. The Teensy publishes its exact target up to its 50 kHz firmware clamp; drops appear only on the receive side once the agent/DDS pipeline saturates.

Drop rate vs target, by payload size (best-effort, C++ subscriber, direct GbE).

| TARGET RATE | POINT32 (12 B) | GALVOSTATE (28 B) |
|---|---|---|
| 20 kHz | **0%** | **0%** |
| 30 kHz | 0.31% | 3.76% |
| 40 kHz | 1.89% | 7.95% |
| 50 kHz | **13.5%** | — |

- **C++ lossless ceiling: ~40 kHz for `Point32`, ~20–25 kHz for the 28-byte `GalvoState`.** The receive path saturates at ~43k msg/s (agent → FastDDS, single-threaded XRCE).
- **Payload size matters past ~12 B.** Going 4 B → 12 B was effectively free — per-message overhead dominated — but the larger multi-signal message measurably lowers the message-rate ceiling.
- **The Teensy is never the bottleneck.** It generates its exact target up to the 50 kHz clamp. The publish loop also resyncs if it falls a full period behind, so a target period near the loop time can't run away into a burst.

## Subscriber matters: Python vs C++

Same firmware, same best-effort 12-byte `Point32` stream; the only change is the subscriber. This isolates the receive-side limiter from the link.

Drop rate vs target rate, Python rclpy meter vs C++ `teensy_bench rate_meter`.

| TARGET RATE | PYTHON RCLPY METER | C++ METER |
|---|---|---|
| 5 kHz | **0%** | **0%** |
| 8 kHz | ~0% (variable) | **0%** |
| 10 kHz | ~6.5% | **0%** |
| 15 kHz | — | **0%** |
| 20 kHz | **~40%** | **0%** |

- **The Python subscriber was the limiter** (~12k msg/s) — it starts shedding samples around 10 kHz and is dropping ~40% by 20 kHz.
- **The C++ subscriber is lossless to 20 kHz** — 40× the 500 Hz goal — and the loss seen in the Python column is the receive side, not the firmware.
- If you only ever read this stream in Python, plan around ~8–10 kHz; for the real ceiling, use a C++ node (or move the high-rate consumer off rclpy).

PART 2 — LATENCY

## Round-trip latency

Measured with `teensy_bench rtt_ping` over a best-effort echo: host → DDS → agent → UDP → Teensy `ping_cb` → UDP → agent → DDS → host. One-way is taken as half the RTT.

Round-trip time vs ping rate (best-effort echo, direct GbE, 0% loss).

| PING RATE | MEDIAN RTT | P99 RTT | ≈ ONE-WAY |
|---|---|---|---|
| 100 Hz | 0.87 ms | 1.60 ms | 0.45 ms |
| 200 Hz | 0.54 ms | 1.17 ms | 0.29 ms |
| 300 Hz | 0.54 ms | 1.18 ms | 0.30 ms |

- **Sub-millisecond RTT (~0.5 ms), ~0.3 ms one-way, p99 < 1.2 ms, 0% loss.** The LAN UDP wire latency itself is sub-ms; the end-to-end figure absorbs the agent, DDS, and WSL hop.
- **The tail can spike** under heavy concurrent telemetry or WSL scheduling jitter, so keep the telemetry rate modest when latency-characterizing. (Part 4 quantifies how the tail moves under host CPU load.)

PART 3 — QOS: BEST-EFFORT VS RELIABLE

## The trade-off, measured

One firmware, two publishers on the same 12-byte payload, isolated per-QoS via `teensy/set_mode` so neither loads the other. Reliable runs over a bounded XRCE stream window ( `RMW_UXRCE_STREAM_HISTORY=16` ).

Received rate / drop rate by QoS and target (5 s windows, drop detection via the `.z` sequence).

| TARGET RATE | BEST-EFFORT: RECEIVED / DROPS | RELIABLE: RECEIVED / DROPS |
|---|---|---|
| 1 kHz | 1000 Hz / **0%** | ~1020 Hz / ~0% |
| 2 kHz | 2000 Hz / **0%** | 2002 Hz / **0%** |
| 3 kHz | ~2824 Hz / ~6% *(varies)* | **capped ~2837 Hz / 0%** |
| 5 kHz | ~5021 Hz / ~5.6% *(varies)* | **capped ~2820 Hz / 0%** |

Best-effort on a *quiet* receive pipeline was lossless to ~5–8 kHz; under load it drops a few percent, and that loss is **variable** run-to-run. The one-line summary: **under rate pressure, best-effort drops data; reliable drops rate.**

- **Best-effort = fire-and-forget.** The Teensy sends every sample; the transport makes no delivery guarantee. Losses happen on the receive side when it can't keep up — lowest latency, highest throughput, but occasional misses, and how many depends on host load (not deterministic).
- **Reliable = acknowledged, in-order delivery** over a bounded stream window. When the Teensy tries to publish faster than acks return, the stream buffer fills and `rcl_publish` **fails on the Teensy side** — the sample is never put on the wire. Whatever is delivered is guaranteed, in order, 0% loss; the effective rate hard-caps (~2.8 kHz here — note both the 3 k and 5 k targets land at ~2.8 kHz); latency is higher (each sample waits on ack round-trips).

> **The reliable ceiling is tunable.** ~2.8 kHz is a function of the ack round-trip × the stream window (`RMW_UXRCE_STREAM_HISTORY`, currently 16). A larger window allows more in-flight unacked samples → higher reliable ceiling, at the cost of RAM and worst-case latency. Raise it in `colcon.meta` and rebuild to push it up. Best-effort has no such window and isn't affected.

## Which to use

| USE CASE | QOS |
|---|---|
| High-rate telemetry / sensor streaming (galvo position, IMU, ADC) | **best-effort** — the newest sample matters most; an occasional miss is fine |
| Commands, setpoints, mode/state changes, e-stop | **reliable** — must not be lost or reordered (low rate, well within the cap) |
| Bulk/critical data that must all arrive | **reliable**, kept under the window-limited ceiling |

This is exactly how the firmware is wired: `galvo_state` telemetry is best-effort; `command` / `set_rate_hz` / `set_mode` are reliable.

QoS caveat: best-effort drop % is load-dependent and varied run-to-run (0% quiet, ~5% busy). These QoS figures were taken through the Python rclpy subscriber, itself a receive-side limiter — a C++ subscriber raises the best-effort ceiling but does not change the reliable window cap, which is set by the XRCE stream + ack RTT on the Teensy/agent side.

PART 4 — ROBUSTNESS UNDER HOST CPU LOAD

## Update rate & latency vs host load

The production host won't be idle — think two cameras at 60 fps 1080p plus OpenCV and galvo control. `host/loadsweep.sh` pegs N of the 16 cores with `yes` busy-loops (sudo-free), then measures the best-effort `galvo_state` rate and round-trip latency at each load level. The agent *and* the meters compete on the same cores — the same topology as production. The load is synthetic and uniform.

### 1 kHz — control-rate target

Best-effort `galvo_state` at a 1 kHz target, vs host CPU load.

| HOST LOAD | RECV RATE | DROPS | RTT P50 | RTT P99 | PING LOSS |
|---|---|---|---|---|---|
| idle | 1000 Hz | **0%** | 0.50 ms | 1.23 ms | 0% |
| 8/16 cores | 1000 Hz | 0.02% | 0.75 ms | 1.43 ms | 0% |
| 15/16 cores | 1001 Hz | 0.15% | 0.93 ms | 5.40 ms | 0% |

### 10 kHz — stress

Best-effort `galvo_state` at a 10 kHz target, vs host CPU load.

| HOST LOAD | RECV RATE | DROPS | RTT P50 | RTT P99 |
|---|---|---|---|---|
| idle | 9998 Hz | **0%** | 0.67 ms | 1.35 ms |
| 8/16 cores | 9997 Hz | ~0% | 0.81 ms | 2.77 ms |
| 15/16 cores | 9152 Hz | **8.2%** | 1.46 ms | 10.7 ms |

- **1 kHz is effectively bulletproof** — 0.15% drops with 15/16 cores pegged, median RTT still sub-millisecond. A control loop here has huge margin.
- **Degradation is graceful, not a cliff.** Heavy load costs a small % of samples and some tail latency — not a rate collapse. (Serial, by contrast, drops to ~100 Hz under load because the Teensy's USB-CDC TX times out when the host can't drain the port; UDP has no such coupling — the kernel socket buffer absorbs it.)
- **The tail moves, the median mostly doesn't.** p99 RTT climbs (1.2 → 5–11 ms) while p50 stays ~1 ms. Expect *occasional* multi-ms spikes under load, not sustained slowdown.
- **10 kHz holds lossless through 8 cores**; only at near-total saturation does it shed ~8%.

**HARDENING THE TAIL UNDER LOAD — PRODUCTION**

1. **Core affinity** — pin cameras/OpenCV to one core set, the agent + control node to another.
2. **Separate the Teensy subscriber from the CV loop** (dedicated thread / executor) so callbacks aren't gated by frame processing.
3. Raise `net.core.rmem_max` / `rmem_default` so bursts never overflow the socket buffer.
4. Best-effort for telemetry, reliable for commands (already the default).

Load caveats: synthetic uniform CPU load — real cameras + OpenCV add memory-bandwidth contention, IRQ load, and burstiness not captured here, so the real rig could be somewhat worse. Linux CFS shares fairly, so even at 15/16 the agent kept ~1 core; aggressive core-pinning by the CV stack could starve it more (hence the pinning guidance above). The authoritative number comes from running `rate_meter` on the real rig with the cameras live.

**APPENDIX — REPRODUCING THE MEASUREMENTS**

All benchmark nodes live in `ros2_ws/src/teensy_bench`; the host load sweep is `microros_ethernet_test/host/loadsweep.sh`. Build once, then drive the rate live over a topic — no reflash needed.

### Build the C++ benchmark nodes

```
cd ros2_ws && colcon build --packages-select teensy_bench && source install/setup.bash
```

### Throughput + drops (best-effort or reliable)

```
# change the publish rate live:
ros2 topic pub -r 10 /teensy/set_rate_hz std_msgs/msg/Int32 "{data: 5000}"
# measure received Hz + drop % over a 5 s window:
ros2 run teensy_bench rate_meter /teensy/galvo_state 5 best_effort
```

### Round-trip latency

```
# rtt_ping <count> <rate_hz>
ros2 run teensy_bench rtt_ping 2000 200
```

### Host-load sweep (rate + RTT at each load level)

```
bash host/loadsweep.sh 1000  "0 8 15"     # control-rate target
bash host/loadsweep.sh 10000 "0 8 15"     # stress
```

`ros2 topic hz` can't measure a best-effort topic — it subscribes reliable and receives nothing. Use the included meters, which match best-effort QoS and report drops via the `z` sequence. The ros2 daemon also caches a stale graph; use `ros2 daemon stop` or `ros2 topic list --no-daemon`.

## Pushing higher, if ever needed

- **Batch N samples per message** (e.g. `Point32[]`) — fewer, larger packets past the per-message overhead wall.
- **Multi-threaded agent**, larger UDP socket buffers, and a higher `RMW_UXRCE_TRANSPORT_MTU` to lift the ~43k msg/s receive ceiling.
- **Overclock the Teensy** (`board_build.f_cpu = 720000000`) — only if the firmware ever caps out, which it currently never does.

---